Reproducible Astrophysics with Nix

Vasilii Pustovoit

Canadian Institute for Theoretical Astrophysics (CITA)

August 15, 2025

For those who want to follow the optional hands-on activity, please install
Nix: the package manager
from nixos.org/download
onto your system.
If you want a headstart, all the activities are located in:
`https://github.com/Vasissualiyp/Nix-Tutorial`

You will need a terminal for this activity!

Astrophysics' Reproducibility Challenge

- "My simulation worked yesterday but fails today!"
- "Can't reproduce my own paper's results"
- "New team member spent a week setting up software"

Astrophysics' Reproducibility Challenge

- "My simulation worked yesterday but fails today!"
- "Can't reproduce my own paper's results"
- "New team member spent a week setting up software"

¯_(ツ)_/¯
IT WORKS
on my machine

"But Conda/Docker/HPC Modules Work For Me!.."

- Conda: Floating dependencies (i.e. what is numpy 2.2?) + host library leaks (are you sure you're using the library you think you're using ?)

"But Conda/Docker/HPC Modules Work For Me!.."

- Conda: Floating dependencies (i.e. what is numpy 2.2?) + host library leaks (are you sure you're using the library you think you're using ?)
- Docker: Good luck with doing HPC (root required, require specific container environments, scheduler issues...); not fully reproducible (host environment leaks)

"But Conda/Docker/HPC Modules Work For Me!.."

- Conda: Floating dependencies (i.e. what is numpy 2.2?) + host library leaks (are you sure you're using the library you think you're using ?)
- Docker: Good luck with doing HPC (root required, require specific container environments, scheduler issues...); not fully reproducible (host environment leaks)
- Modules: Configuration drift over time (module load python on CITA cluster... THX SYSADMINS)

"But Conda/Docker/HPC Modules Work For Me!.."

- Conda: Floating dependencies (i.e. what is numpy 2.2?) + host library leaks (are you sure you're using the library you think you're using ?)
- Docker: Good luck with doing HPC (root required, require specific container environments, scheduler issues...); not fully reproducible (host environment leaks)
- Modules: Configuration drift over time (module load python on CITA cluster... THX SYSADMINS)

All fail at bit-for-bit reproducibility across systems/time

Nix: A Paradigm Shift

Nix is:

- build system
- package manager
- Linux-based OS
- programming language

Nix: A Paradigm Shift

Nix is:

- build system
- package manager
- Linux-based OS
- programming language

A system that focuses on construction of **declarative and reproducable** software environments.

Why Nix?

- **Pure (Isolated) Builds**

  No host leaks - strict sandboxing

- **Cryptographic Hashing**

  Unique IDs for *complete* dependency trees

- **Declarative Environments**

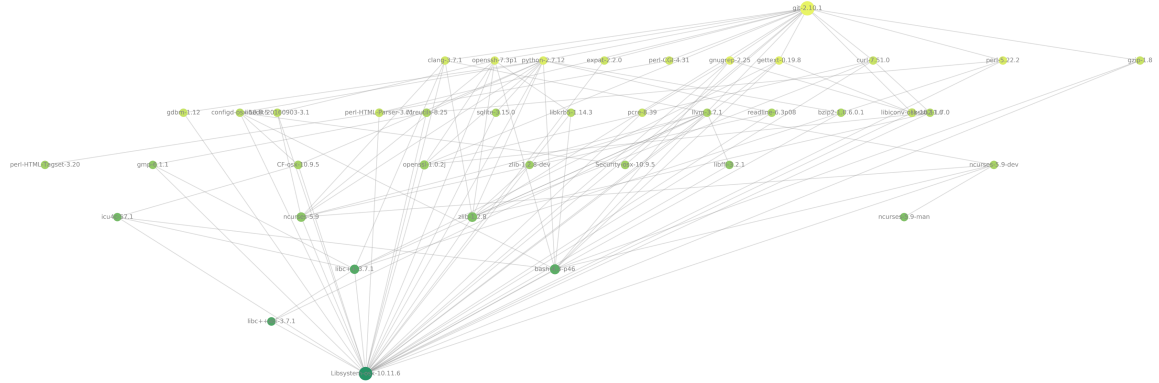
  Define what you want, not how to build it

- **Atomic Operations**

  Instant rollbacks, no broken states

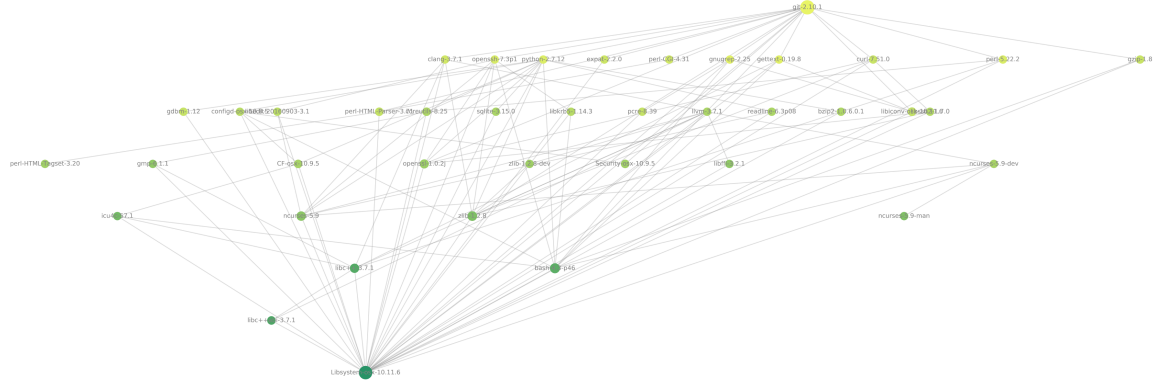
Why build environment isolation?

Dependency graph:



Why build environment isolation?

Dependency graph:



Environment purity (isolation) prevents dependency on an external (host, internet) state!

Why hashing?

Each built of a package gets a cryprographic hash, based on:

- **Build parameters** (i.e. compiler version and flags, environment variables...)

Why hashing?

Each built of a package gets a cryprographic hash, based on:

- **Build parameters** (i.e. compiler version and flags, environment variables...)
- **Specified package parameters** (i.e. download link, package metadata...)

Why hashing?

Each built of a package gets a cryprographic hash, based on:

- **Build parameters** (i.e. compiler version and flags, environment variables...)
- **Specified package parameters** (i.e. download link, package metadata...)
- **Dependencies' hashes** (to avoid dependency hell)

Why hashing?

Each built of a package gets a cryprographic hash, based on:

- **Build parameters** (i.e. compiler version and flags, environment variables...)
- **Specified package parameters** (i.e. download link, package metadata...)
- **Dependencies' hashes** (to avoid dependency hell)

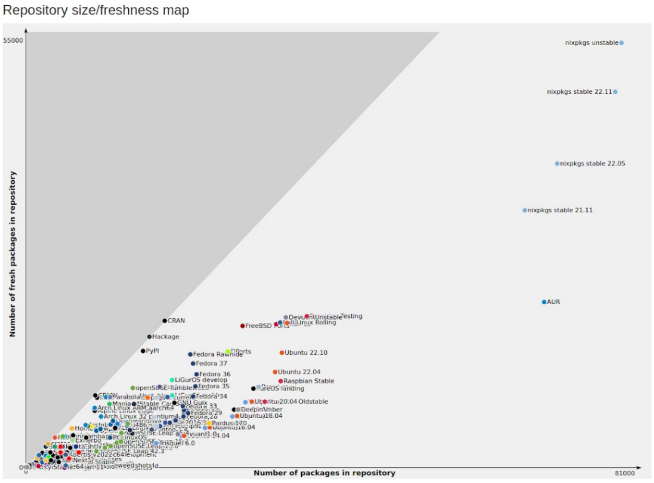
This means that if 2 builds of some package have the same hash, they are guaranteed to be built in an identical way! $\rightarrow$ **functionally identical builds**

Identical Builds = Easy Packaging

Since it is easy to produce identical builds, software that you package will live longer.

Identical Builds = Easy Packaging

Since it is easy to produce identical builds, software that you package will live longer.



Nixpkgs

Nixpkgs is a FOSS package repository that uses Nix language and Nix build system to package software. Anyone can contribute!

Nixpkgs

Nixpkgs is a FOSS package repository that uses Nix language and Nix build system to package software. Anyone can contribute!
When you are creating your environment, you usually pull the packages and dependencies from nixpkgs.

Example: PeakPatch

Example: PeakPatch

- Pre-Nix: could only run on Niagara
- Post-Nix: reproducible on any machine with Nix

Dependencies include:

- GCC (GNU Compilers for C, Fortran)
- GSL (GNU Scientific Library)
- OpenMP (Parallelism)
- OpenMPI (Parallelism)
- FFTW (both single and double precision with MPI support)
- HDF5
- And 10+ Python packages (Including nontrivially-packaged)

Example: PeakPatch

- Pre-Nix: could only run on Niagara
- Post-Nix: reproducible on any machine with Nix

Dependencies include:

- GCC (GNU Compilers for C, Fortran)
- GSL (GNU Scientific Library)
- OpenMP (Parallelism)
- OpenMPI (Parallelism)
- FFTW (both single and double precision with MPI support)
- HDF5
- And 10+ Python packages (Including nontrivially-packaged)

All can be activated on any Nix-enabled machine with a single command!

```
{
  description = "PeakPatch developement environment";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    flake-utils.url = "github:numtide/flake-utils";
  };
  outputs = { self, nixpkgs, flake-utils, ... }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = import nixpkgs {
          inherit system;
          config.allowUnfree = true;
        };
        customFftw_single = # ... package override (for MPI)
        customFftw_double = # ... package override (for MPI)
        python = pkgs.python311Packages;
        camb = python.buildPythonPackage rec {
          # package declarations (not in nixpkgs)
        };
        class = python.buildPythonPackage rec {
          # package declarations (not in nixpkgs)
        };
    # ... More packages ...
```

```
        pythonEnv = (python.python.withPackages (ps: with ps; [
          pandas
          matplotlib
          numpy
          astropy
          # ... More python packages
        ]));
      in
      {
        devShell = pkgs.mkShell {
          buildInputs = with pkgs; [
            pythonEnv
            gsl
            gcc
            llvmPackages.openmp
            customFftw_single
            customFftw_double
            # ... More packages
          ];
          shellHook = ''
            export PP_DIR=$(pwd)
      # ... More environment variables and welcome message
            '';
        };
      }
    );
}
```

NixOS

NixOS is a logical extension of philosophy of Nix to an operating system customization. Reproducibility extends to inter-version and inter-machine reproducibility!

NixOS

NixOS is a logical extension of philosophy of Nix to an operating system customization.
Reproducibility extends to inter-version and inter-machine reproducibility!

Example:

You updated your system, and a new version of Zoom doesn't capture your screen.
NixOS: Roll back to previos version in 30 sec

NixOS

NixOS is a logical extension of philosophy of Nix to an operating system customization.
Reproducibility extends to inter-version and inter-machine reproducibility!

Example:

You updated your system, and a new version of Zoom doesn't capture your screen.
NixOS: Roll back to previos version in 30 sec

Example:

Your laptop physically broke, but all your data is backed up.
NixOS: Re-install your system on a new machine with your config file

Too many words.

Too many words.

What does this mean for me?

Too many words.

What does this mean for me?

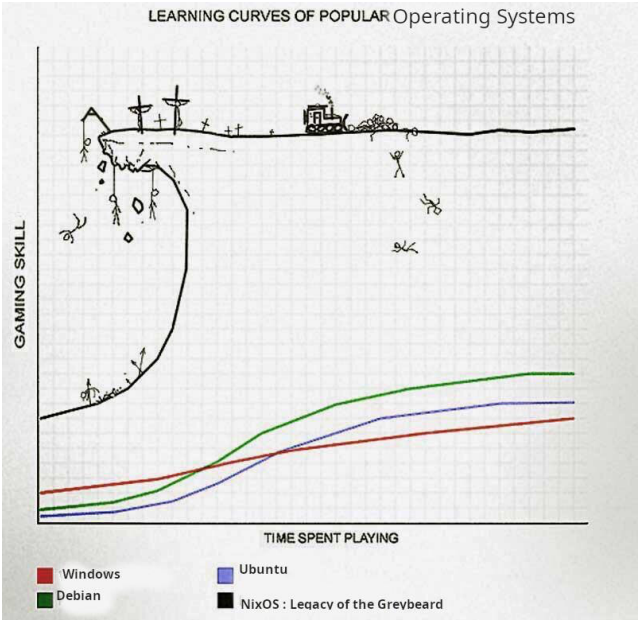
With Nix, reproducibility of A LOT OF software is easy!*

Too many words.

What does this mean for me?

With Nix, reproducibility of A LOT OF software is easy!*

If you are ready to pay the price...


LEARNING CURVES OF POPULAR Operating Systems
GAMING SKILL
TIME SPENT PLAYING
Windows
Debian
Ubuntu
NixOS : Legacy of the Greybeard

When to Use Nix (and When Not To)

Strengths:

- Long-term reproducibility
- HPC environment control
- Multi-project collaboration
- Cross-platform consistency
- No dependency hell

Challenges:

- Learning curve
- Proprietary software ($\pm$)
- IDE integration ($\pm$)
- No POSIX compliance by default (It's actually good)
- Storage overhead (10-50GB) (depending on setup)

Getting Started

- zero-to-nix.com
- nixos.org
- nixos.org/guides/nix-pills
- discourse.nixos.org

Conclusion

- Nix solves deep reproducibility where others fail
- Essential for long-term astrophysics projects
- Steep initial learning curve, long-term payout

Conclusion

- Nix solves deep reproducibility where others fail
- Essential for long-term astrophysics projects
- Steep initial learning curve, long-term payout

Thank You!

References I

Learning curve